1.3 Vector Addition and Subtraction

Core skills you will master in this section:

► Basic definitions and rules for vector addition and subtraction: addition sums corresponding components, while subtraction adds the opposite vector.

► Geometric perspectives on the addition rule: the parallelogram law and the triangle law.

► The commutative and associative laws of vector addition: understanding that different orders of addition do not affect the result, and using diagrams to see that the addition path is invariant.

► The parallelepiped law: understanding how three vectors combine in space, and grasping its relationship to the parallelogram law.

► Using NumPy to compute vector addition and subtraction, and using Matplotlib to visualize them.

Vector addition and subtraction are among the core operations of linear algebra; this section introduces their basic definitions, rules, and key properties. From a geometric perspective, we introduce the parallelogram law and the triangle law, each used to understand the geometric meaning of vector addition. The parallelogram law represents the sum of two vectors by constructing a parallelogram, while the triangle law finds the sum by connecting vectors head to tail. In addition, for three linearly independent vectors in three-dimensional space, addition can be represented intuitively using the parallelepiped law, which applies to the addition of three vectors and illustrates the geometric meaning of the associative law. In the vector subtraction section, we understand subtraction as the process of adding the opposite vector.

Vector Addition

For two vectors of the same dimension, vector addition means adding their corresponding components to obtain a new vector.

For example, given the following two n-dimensional column vectors

$$\boldsymbol{a}=\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}, \quad \boldsymbol{b}=\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

The sum of vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ is obtained by adding their corresponding components; the result is a vector of the same dimension,

$$\boldsymbol{a}+\boldsymbol{b}=\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}+\begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}=\begin{bmatrix} a_1+b_1 \\ a_2+b_2 \\ \vdots \\ a_n+b_n \end{bmatrix}$$

The code below demonstrates how to perform vector addition with NumPy. The code is simple, so please annotate it yourself as you study it.

Code 1. Vector Addition | LA_01_03_01.ipynb

```
## 初始化
import numpy as np

## 定义向量
a_vec = np.array([4, 1])
b_vec = np.array([1, 2])

## 向量加法
a_plus_b = a_vec + b_vec
```

Using RGB Color Vectors as an Example

In the RGB color model, the red vector $\boldsymbol{e}_1$, green vector $\boldsymbol{e}_2$, and blue vector $\boldsymbol{e}_3$ represent the three primary colors. Vector addition can be intuitively understood as color mixing.

As shown in Figure 1, we can use the rules of vector addition to obtain new colors through different combinations.

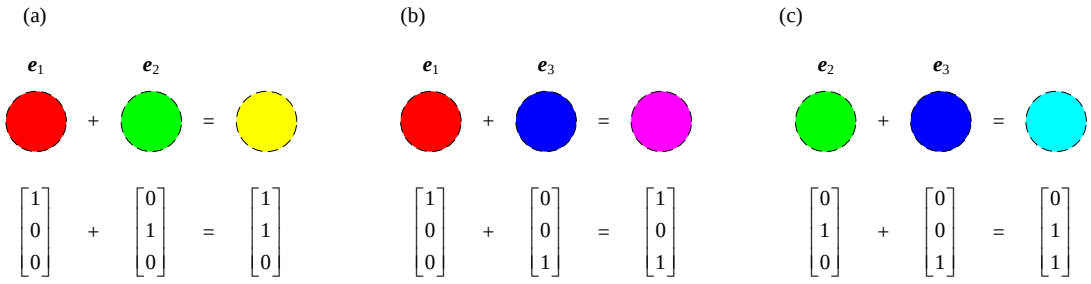

Figure 1. Vector Addition: Mixing Two Primary Colors

For example, as shown in Figure 1(a), the red vector $\boldsymbol{e}_1$ and green vector $\boldsymbol{e}_2$ sum to give the yellow vector, i.e.,

$$\boldsymbol{e}_1 + \boldsymbol{e}_2 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix}$$

As shown in Figure 1(b), the red vector $\boldsymbol{e}_1$ and blue vector $\boldsymbol{e}_3$ sum to give magenta; as shown in Figure 1(c), the green vector $\boldsymbol{e}_2$ and blue vector $\boldsymbol{e}_3$ sum to give cyan.

Please write out the addition process for these two vector pairs yourself.

As shown in Figure 2, when we add the red vector $\boldsymbol{e}_1$, green vector $\boldsymbol{e}_2$, and blue vector $\boldsymbol{e}_3$ all together, i.e., $\boldsymbol{e}_1 + \boldsymbol{e}_2 + \boldsymbol{e}_3$, we get the white vector, i.e.,

$$\boldsymbol{e}_1+\boldsymbol{e}_2+\boldsymbol{e}_3=\begin{bmatrix}1\\0\\0\end{bmatrix}+\begin{bmatrix}0\\1\\0\end{bmatrix}+\begin{bmatrix}0\\0\\1\end{bmatrix}=\begin{bmatrix}1\\1\\1\end{bmatrix}$$

This means that in RGB color space, mixing the light of the three primary colors produces white light. This operation not only follows the rules of vector addition but also illustrates the principle of additive light mixing in the RGB color model.

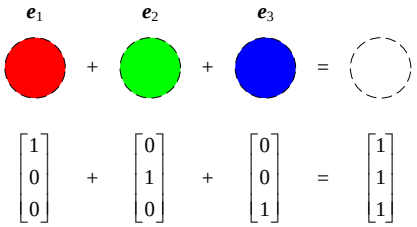

Figure 2. Vector Addition: Mixing Three Primary Colors

Geometric Perspective: The Parallelogram Law

The parallelogram law is a geometric method for computing the sum of two vectors.

This method aligns the starting points of the two vectors and uses them as adjacent sides to construct a parallelogram; the diagonal from the starting point (the origin) to the opposite vertex gives the result of the vector addition.

Figure 3 shows the sum of two vectors solved using the parallelogram law, where the nonzero vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ in each subplot exhibit different relationships.

Figure 3 (a) is a unit square; Figure 3 (b) is a square; Figure 3 (c) is a rectangle; Figure 3 (d) has two sides of the parallelogram parallel to the horizontal axis; Figure 3 (e) has two sides parallel to the vertical axis.

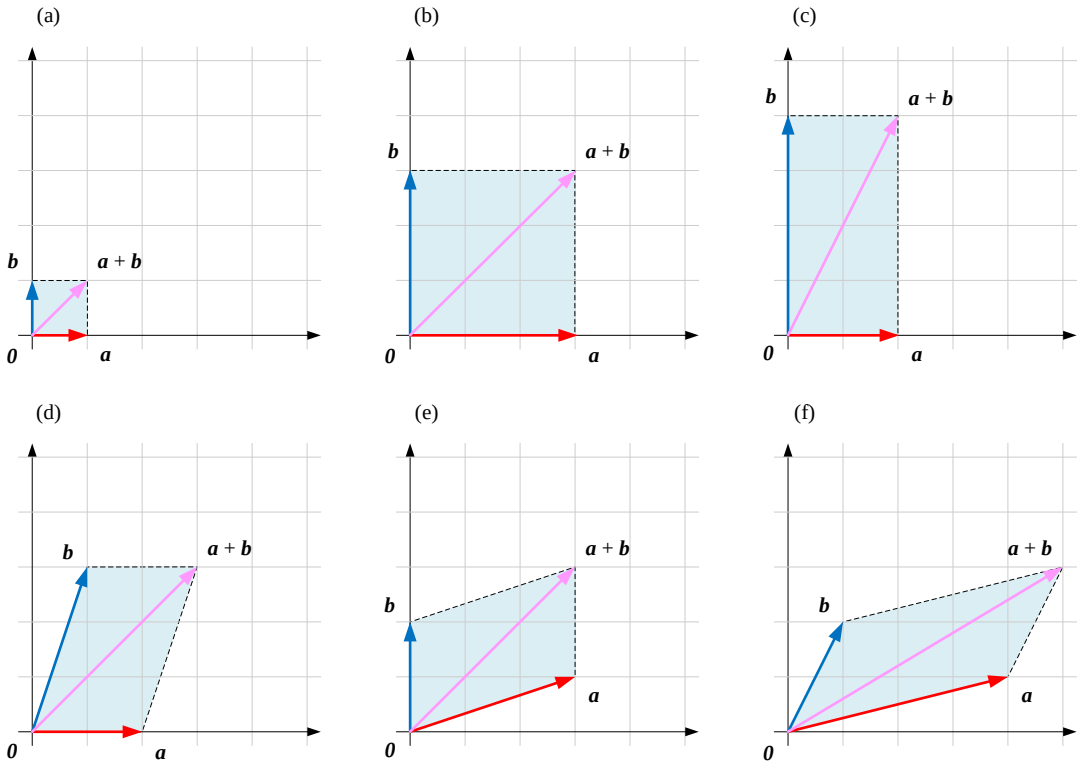

Figure 3. A Few Examples of the Parallelogram Law

In particular, as shown in Figure 4, if the nonzero vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ are collinear, meaning they lie on the same line, $\boldsymbol{a}$ and $\boldsymbol{b}$ point in the same or opposite directions. Their sum $\boldsymbol{a} + \boldsymbol{b}$ also lies on this line, so no parallelogram can be formed.

In addition, we have repeatedly emphasized the case of nonzero vectors. If either vector $\boldsymbol{a}$ or $\boldsymbol{b}$ is the zero vector $\boldsymbol{0}$, then their sum still lies on the line containing the nonzero vector, and no parallelogram can be formed.

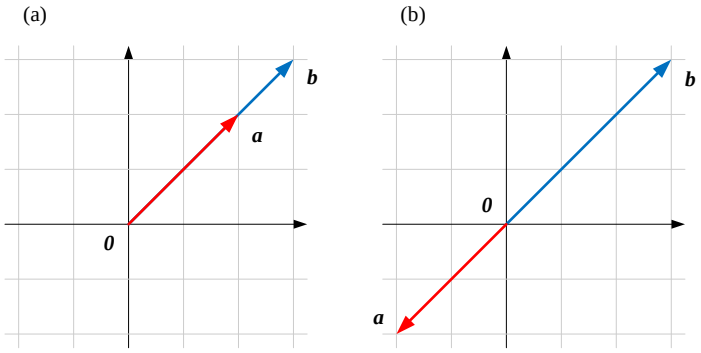

Figure 4. $\boldsymbol{a}$ and $\boldsymbol{b}$ Collinear

Code 2 visualizes vector addition using the parallelogram law. Let's walk through the key lines below.

a This imports two tools, numpy and matplotlib.pyplot; numpy is mainly used for array and vector computation, while matplotlib.pyplot is used for plotting figures.

b The numpy.array() function is used to define two vectors, a_vec and b_vec.

c This computes the sum of vectors a_vec and b_vec and stores it in the variable a_plus_b. This calculation relies on numpy's elementwise addition of arrays, meaning a_vec + b_vec automatically adds the corresponding components of the two arrays.

d This calls plt.figure(figsize=(6,6)), which creates a new plotting window and sets the figure size. The parameter figsize=(6,6) means the figure's width and height are both 6 inches.

e plt.quiver() is used to draw the first vector, a_vec.

The function plt.quiver(x, y, u, v, ...) is used to draw vectors; the parameters (x, y) specify the vector's starting point, here (0,0), meaning it starts from the origin.

The parameters (u, v) represent the vector's direction and length, namely a_vec[0] and a_vec[1], i.e., the vector's x and y components.

angles='xy' means the vector is drawn according to the standard 2D coordinate system.

scale_units='xy' scales the vector's length according to the axis tick marks.

scale=1 keeps the vector at its actual size, without further scaling.

color='r' makes the vector red, and label="a" labels it as a in the legend.

f The code for drawing vector b_vec is e almost identical to this.

g This plots the vector a_plus_b, i.e., the resulting vector of a_vec + b_vec.

To help observe the geometric properties of vector addition, h uses plt.plot() to draw a dashed auxiliary line for the parallelogram.

plt.plot(x_values, y_values, linestyle) is used to draw a line connecting (x_values[0], y_values[0]) and (x_values[1], y_values[1]).

The x-coordinates of the first auxiliary line are [b_vec[0], a_plus_b[0]], i.e., [1,5], and the y-coordinates are [b_vec[1], a_plus_b[1]], i.e., [2,3]; this line connects the endpoint of b_vec to the endpoint of a_plus_b.

'k--' denotes a black dashed line.

i This draws the other auxiliary line of the parallelogram.

j In the lines that follow, this code is used to format the plot.

plt.gca().set_aspect('equal', adjustable='box') keeps the axis proportions consistent, ensuring the same scaling in the x and y directions; otherwise the figure might appear stretched.

plt.xlim(0, 5) sets the x-axis range to [0,5], and plt.ylim(0, 5) sets the y-axis range to [0,5], so the entire figure is fully displayed within this range.

plt.axhline(0, color='black', linewidth=0.5) and plt.axvline(0, color='black', linewidth=0.5) draw the baselines for the x-axis and y-axis, in black with a line width of 0.5.

plt.grid(color='gray', alpha=0.8, linestyle='-', linewidth=0.25) displays gray gridlines in the background to make the figure clearer; alpha=0.8 gives the gridlines some transparency, linestyle='-' makes them solid lines, and linewidth=0.25 keeps them thin.

Finally, plt.legend() displays the legend; it automatically uses the names specified by the label parameter in plt.quiver() to help distinguish the different vectors.

Code 2. The Parallelogram Law | LA_01_03_02.ipynb

```python
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 定义向量
a_vec = np.array([4, 1])
b_vec = np.array([1, 2])

## 计算向量和
a_plus_b = a_vec + b_vec

## 可视化
plt.figure(figsize=(6,6))

# 绘制向量 a
plt.quiver(0, 0, a_vec[0], a_vec[1],
           angles='xy', scale_units='xy',
           scale=1, color='r', label="a")

# 绘制向量 b
plt.quiver(0, 0, b_vec[0], b_vec[1],
           angles='xy', scale_units='xy',
           scale=1, color='b', label="a")

# 绘制向量 a + b
plt.quiver(0, 0, a_plus_b[0], a_plus_b[1],
           angles='xy', scale_units='xy',
           scale=1, color='pink', label="a + b")

# 画出平行四边形的另外两条边
plt.plot([b_vec[0], a_plus_b[0]],
         [b_vec[1], a_plus_b[1]], 'k--')

plt.plot([a_vec[0], a_plus_b[0]],
         [a_vec[1], a_plus_b[1]], 'k--')

# 装饰
plt.gca().set_aspect('equal', adjustable='box')
plt.xlim(0, 5)
plt.ylim(0, 5)
plt.axhline(0, color='black', linewidth=0.5)
plt.axvline(0, color='black', linewidth=0.5)
plt.grid(color='gray', alpha=0.8, linestyle='-', linewidth=0.25)
plt.legend()
```

The Parallelogram Law in RGB Color Mixing

In RGB color space, we can use the parallelogram law to intuitively understand the addition of the three vectors in Figure 1.

As shown in Figure 5 (a), we have the red vector $\boldsymbol{e}_1$ and green vector $\boldsymbol{e}_2$, whose starting points are aligned at the origin.

According to the parallelogram law, we can take the red vector $\boldsymbol{e}_1$ and green vector $\boldsymbol{e}_2$ as two adjacent sides of a parallelogram, then draw the other two sides starting from their starting points. The diagonal of the parallelogram points from the starting point toward the sum, giving us the yellow vector $\boldsymbol{e}_1 + \boldsymbol{e}_2$.

Similarly, we can use the parallelogram law to obtain magenta and cyan.

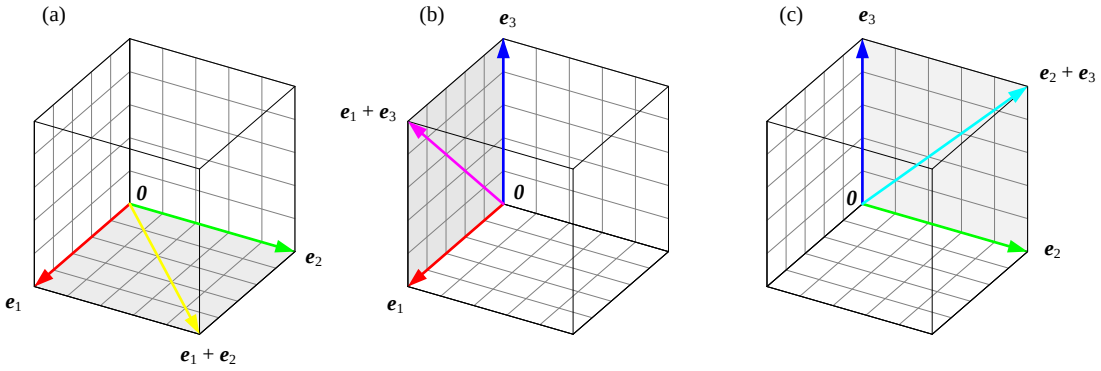

Figure 5. The Parallelogram Law: Mixing Two Primary Colors

The zero vector $\boldsymbol{0}$ is the additive identity for vector addition, satisfying

$$\boldsymbol{a} + \boldsymbol{0} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix}$$

The additive identity is the special vector that, when added to any vector, leaves that vector unchanged.

The equation above expresses that adding the zero vector $\boldsymbol{0}$ to any vector leaves the result unchanged.

Geometrically, the zero vector $\boldsymbol{0}$ can be viewed as the starting point of addition, meaning every vector sum can be thought of as starting from the origin $\boldsymbol{0}$ and accumulating step by step along different directions.

For example, in RGB color space, the zero vector $\boldsymbol{0}$ corresponds to black (i.e., no light at all). If we start from black $\boldsymbol{0}$ and successively add the red vector $\boldsymbol{e}_1$ and green vector $\boldsymbol{e}_2$, we eventually arrive at the yellow vector.

This way of thinking intuitively shows that addition is a process that continually advances from the origin, with the zero vector providing the starting position for this process.

The Parallelepiped

The addition of three vectors can be understood geometrically as constructing a parallelepiped. Geometrically, a parallelepiped is a solid formed by six parallelograms.

Suppose we have three vectors, each starting from the origin. We can first compute the sum of the first two vectors and locate their combined vector in space. Then we add the third vector to this result, and the final endpoint represents the sum of all three vectors. Geometrically, this forms a parallelepiped in three-dimensional space. This construction extends naturally beyond three vectors:

the sum of any number of vectors, in a space of any dimension, can always be found by repeatedly applying the parallelogram (or triangle) law one vector at a time.

Figure 6 shows different cases of adding three vectors, where the relationship among the three vectors differs slightly in each subplot.

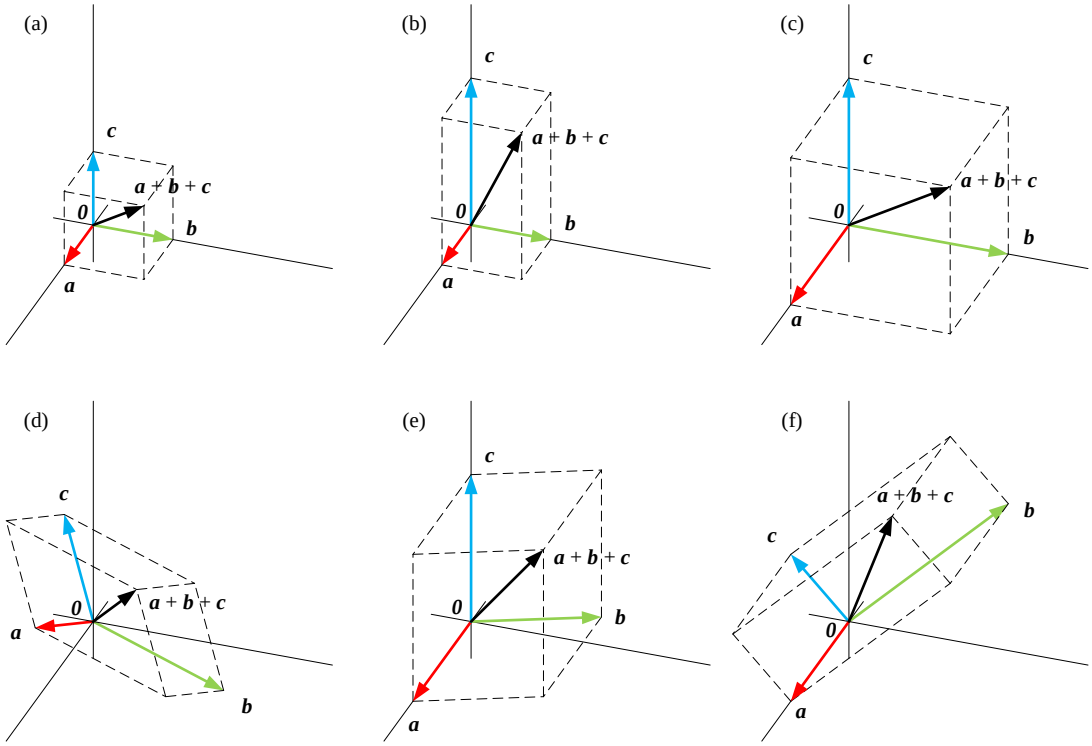

Figure 6. Understanding the Addition of Three (Non-Coplanar) Vectors via the Parallelepiped

There are, of course, exceptions. As shown in Figure 7 (a), if the three vectors happen to lie in the same plane, i.e., they are coplanar, the geometric structure formed by their sum degenerates into a parallelogram rather than forming a true three-dimensional parallelepiped.

In particular, as shown in Figure 7 (b), if the three vectors connect head to tail to form a closed triangle, then their sum is exactly the zero vector $\boldsymbol{0}$; this means that starting from the point $\boldsymbol{0}$ and moving in the order of the vectors, we eventually return to the starting point $\boldsymbol{0}$.

This is, in fact, an instance of the triangle law, a topic we will turn to shortly.

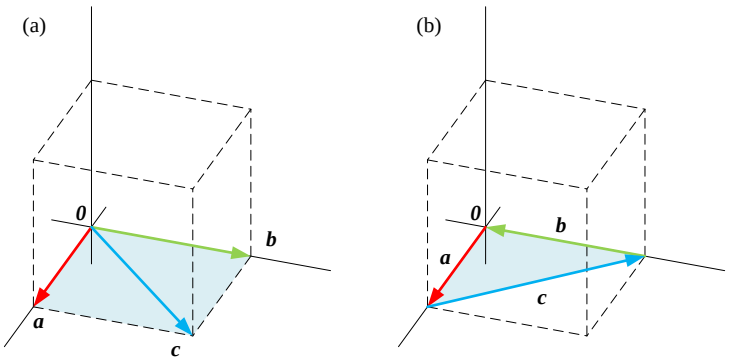

Figure 7. Three Coplanar Vectors

The Parallelepiped Law in RGB Color Mixing

Back to RGB color space. As shown in Figure 8, the red vector $\boldsymbol{e}_1$, green vector $\boldsymbol{e}_2$, and blue vector $\boldsymbol{e}_3$ start from the zero vector $\boldsymbol{0}$ and correspond to the three adjacent edges of the parallelepiped.

The sum of these three vectors $\boldsymbol{e}_1 + \boldsymbol{e}_2 + \boldsymbol{e}_3$ corresponds to the diagonal of the parallelepiped, which shares its starting point with the three vectors and ends at the opposite vertex of the parallelepiped. It's worth noting that in Figure 8, this parallelepiped is a unit cube.

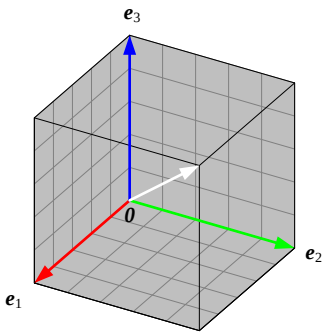

Figure 8. The Parallelepiped (Unit Cube): Computing the Sum of the Red Vector $\boldsymbol{e}_1$, Green Vector $\boldsymbol{e}_2$, and Blue Vector $\boldsymbol{e}_3$

Two Parallelograms

For the addition of three non-coplanar vectors, the parallelepiped law is in fact built from two applications of the parallelogram law.

Specifically, we first use the parallelogram law to add two of the vectors, obtaining a new vector; then, using this new vector and the third vector as sides, we apply the parallelogram law again to obtain the sum of all three vectors. In this way, the parallelepiped law is really two successive applications of the parallelogram law.

As shown in Figure 9, the sum of the red vector $\boldsymbol{e}_1$, green vector $\boldsymbol{e}_2$, and blue vector $\boldsymbol{e}_3$ can be decomposed via two applications of the parallelogram law.

First, we use the red vector $\boldsymbol{e}_1$ and green vector $\boldsymbol{e}_2$ to construct the first parallelogram; its diagonal is their sum, the yellow vector $\boldsymbol{e}_1 + \boldsymbol{e}_2$.

Next, taking the yellow vector $\boldsymbol{e}_1 + \boldsymbol{e}_2$ and the blue vector $\boldsymbol{e}_3$ as sides, we again apply the parallelogram law to construct the second parallelogram. This time, the diagonal is the total sum of all three vectors, i.e., the white vector $(\boldsymbol{e}_1 + \boldsymbol{e}_2) + \boldsymbol{e}_3$.

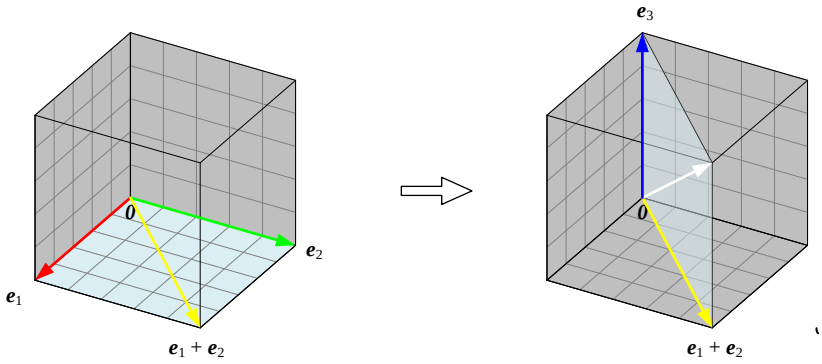

Figure 9. Decomposing the Parallelepiped into Two Parallelograms, $(e_1 + e_2) + e_3$

Geometric Perspective: The Triangle Law

The triangle law is a geometric method for computing the sum of two vectors. It connects the starting point of the second vector to the endpoint of the first vector; the directed line segment from the starting point of the first vector to the endpoint of the second vector is the result of the vector addition. In fact, the triangle law and the parallelogram law describe the same operation from two complementary views: the parallelogram's two adjacent sides and its diagonal correspond exactly to two sides and the closing side of the triangle formed once one vector is translated to the tip of the other.

Figure 3 shows additions computed using the parallelogram law; these can equally be computed using the triangle law, as shown in Figure 10.

As before, when the vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ are collinear, they lie on the same line; regardless of whether they point in the same or opposite directions, their sum still lies on that line, so no triangle can be formed.

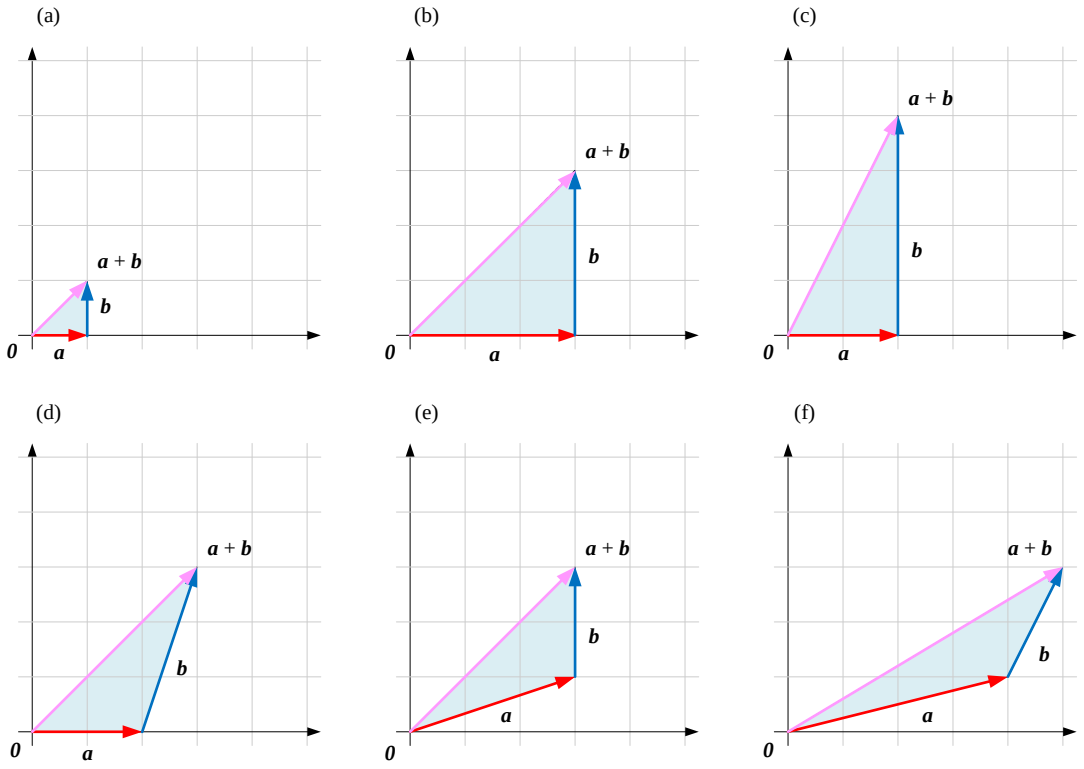

Figure 10. A Few Examples of Computing the Vector Sum $\boldsymbol{a}+\boldsymbol{b}$ Using the Triangle Law

The code file LA_01_03_03.ipynb visualizes vector addition using the triangle law; the code is similar to LA_01_03_02.ipynb, so please study it on your own.

The Triangle Law in RGB Color Mixing

In RGB color space, vector addition can be intuitively understood through the triangle law.

For example, suppose we want to add the red vector $\boldsymbol{e}_1$ and green vector $\boldsymbol{e}_2$ to obtain the yellow vector.

According to the triangle law, we can move the starting point of the green vector $\boldsymbol{e}_2$ to the endpoint of the red vector $\boldsymbol{e}_1$. Once connected head to tail, the directed line segment from the starting point of the red vector $\boldsymbol{e}_1$ to the endpoint of the green vector $\boldsymbol{e}_2$ is the yellow vector.

Equivalently, we can instead move the starting point of the red vector $\boldsymbol{e}_1$ to the endpoint of the green vector $\boldsymbol{e}_2$, and we still arrive at the same yellow vector.

This, in fact, reflects the commutative law of vector addition, namely

$$\boldsymbol{e}_1+\boldsymbol{e}_2=\boldsymbol{e}_2+\boldsymbol{e}_1$$

Similarly, we can use the triangle law to intuitively understand the addition of vectors for other colors.

For example, to add the red vector $\boldsymbol{e}_1$ and blue vector $\boldsymbol{e}_3$, we can first move the starting point of the blue vector $\boldsymbol{e}_3$ to the endpoint of the red vector $\boldsymbol{e}_1$; the resulting directed line segment is the magenta vector.

Alternatively, we can first move the starting point of the red vector $\boldsymbol{e}_1$ to the endpoint of the blue vector $\boldsymbol{e}_3$, which again gives the magenta vector — another illustration of the commutative law of vector addition.

Similarly, when adding the green vector $\boldsymbol{e}_2$ and blue vector $\boldsymbol{e}_3$, we can proceed the same way; whichever vector we move first, we end up with the cyan vector.

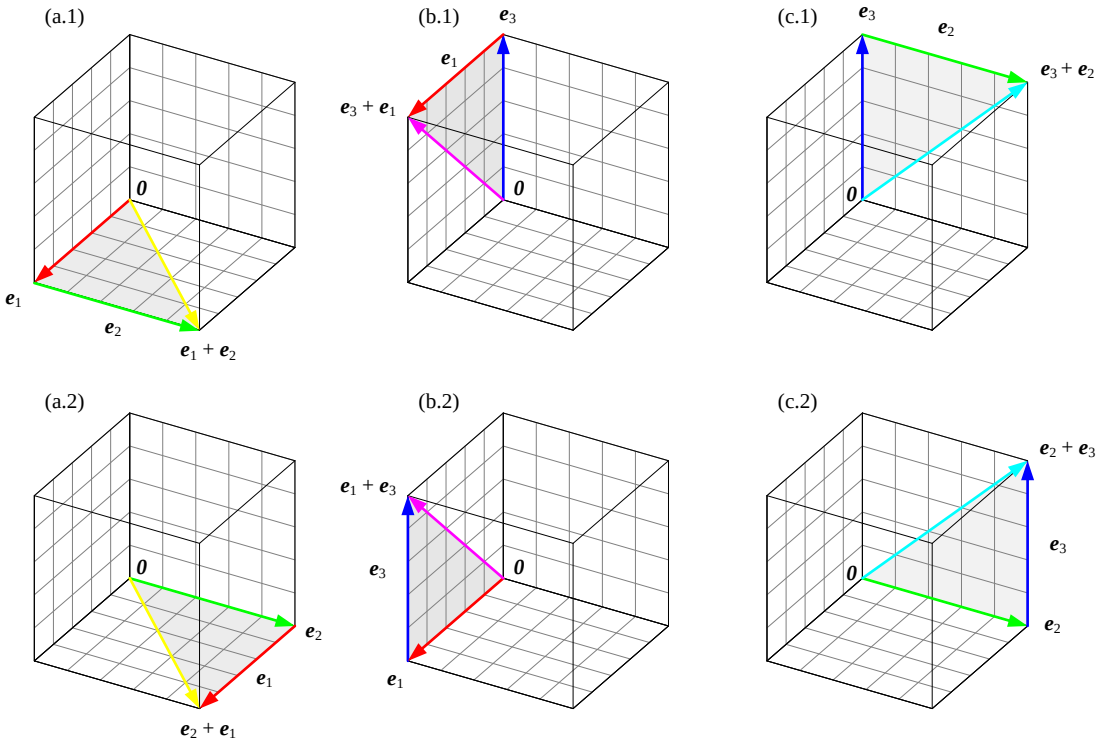

Figure 11. The Commutative Law of Vector Addition

Adding Multiple Vectors: Connecting Head to Tail in Sequence

Geometrically, the triangle law can also be understood as a process of connecting vectors head to tail.

Suppose we have three non-coplanar vectors connected head to tail in sequence: the endpoint of the first vector becomes the starting point of the second, and the endpoint of the second becomes the starting point of the third. This ultimately produces a combined vector running from the initial starting point to the final endpoint.

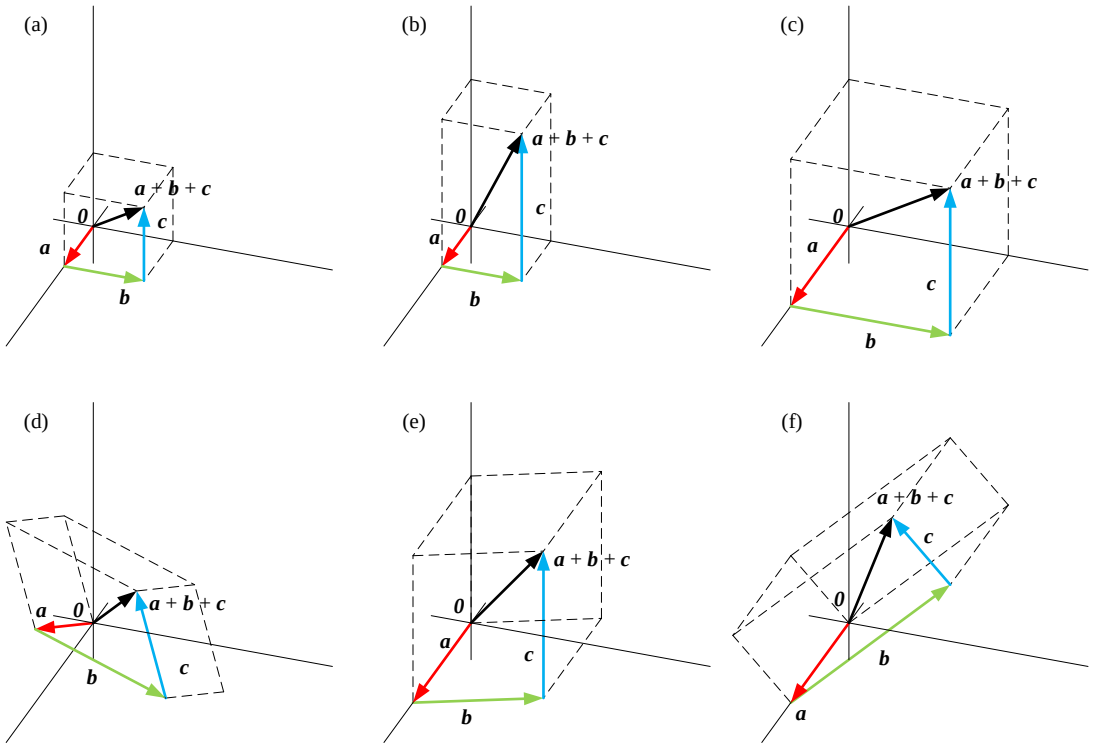

Figure 12. Adding Three Vectors Using the Head-to-Tail Triangle Law

For example, in RGB color space, if the red vector $\boldsymbol{e}_1$, green vector $\boldsymbol{e}_2$, and blue vector $\boldsymbol{e}_3$ are connected head to tail in sequence according to the triangle law, the final vector points to white, i.e., $\boldsymbol{e}_1 + \boldsymbol{e}_2 + \boldsymbol{e}_3$.

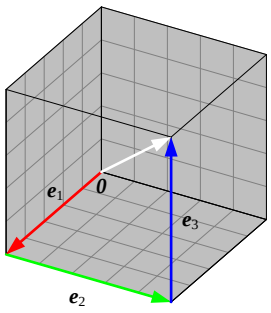

Figure 13. Computing the Sum of the Red Vector $\boldsymbol{e}_1$, Green Vector $\boldsymbol{e}_2$, and Blue Vector $\boldsymbol{e}_3$ by Connecting Head to Tail in Sequence

Using the associative law of vector addition introduced earlier, the sum $\boldsymbol{e}_1 + \boldsymbol{e}_2 + \boldsymbol{e}_3$ can be written as two different paths, as shown in Figure 14.

The first path is $(\boldsymbol{e}_1 + \boldsymbol{e}_2) + \boldsymbol{e}_3$; we first compute $(\boldsymbol{e}_1 + \boldsymbol{e}_2)$ to get the yellow vector, then add the blue vector $\boldsymbol{e}_3$, and finally obtain the white vector.

The second path is $\boldsymbol{e}_1 + (\boldsymbol{e}_2 + \boldsymbol{e}_3)$; we first compute $(\boldsymbol{e}_2 + \boldsymbol{e}_3)$ to get the cyan vector, then add the red vector $\boldsymbol{e}_1$, likewise obtaining the white vector.

Geometrically, this also means that the path of vector composition can differ, while the endpoint remains the same.

This method illustrates the associative law of vector addition, namely:

$$(\boldsymbol{e}_1+\boldsymbol{e}_2)+\boldsymbol{e}_3=\boldsymbol{e}_1+(\boldsymbol{e}_2+\boldsymbol{e}_3)$$

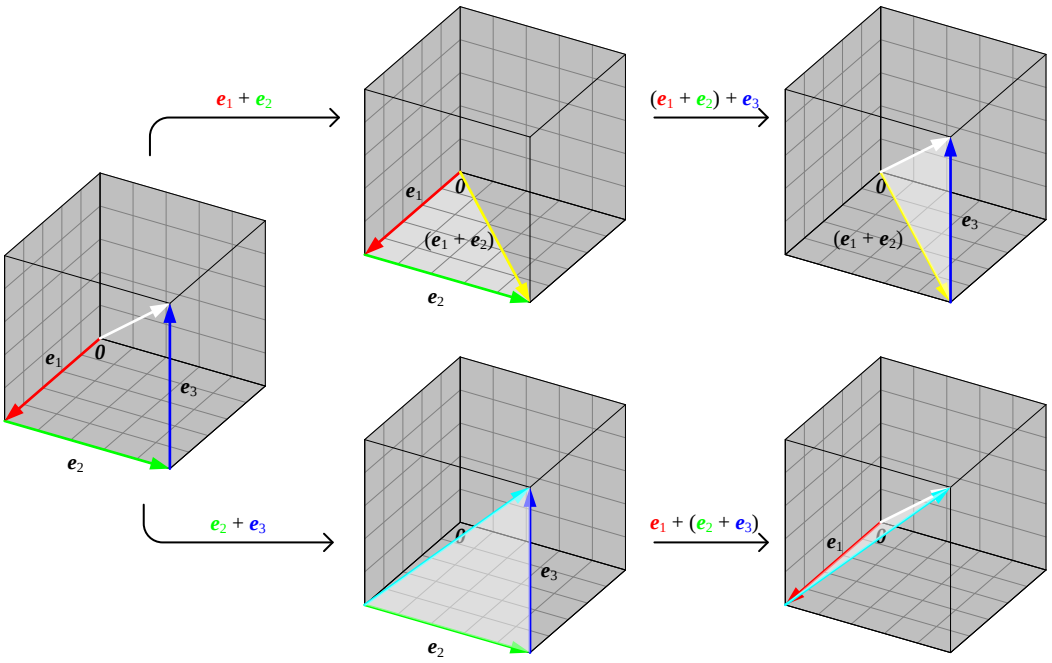

Figure 14. The Two Paths $(\boldsymbol{e}_1 + \boldsymbol{e}_2) + \boldsymbol{e}_3$ and $\boldsymbol{e}_1 + (\boldsymbol{e}_2 + \boldsymbol{e}_3)$

Combining the commutative and associative laws, we can obtain different paths to reach the white vector.

As shown in Figure 15, to obtain the white vector, we can use $\boldsymbol{e}_3 + \boldsymbol{e}_2 + \boldsymbol{e}_1$. This sum can also be written in two forms, namely the two paths $(\boldsymbol{e}_3 + \boldsymbol{e}_2) + \boldsymbol{e}_1$ and $\boldsymbol{e}_3 + (\boldsymbol{e}_2 + \boldsymbol{e}_1)$.

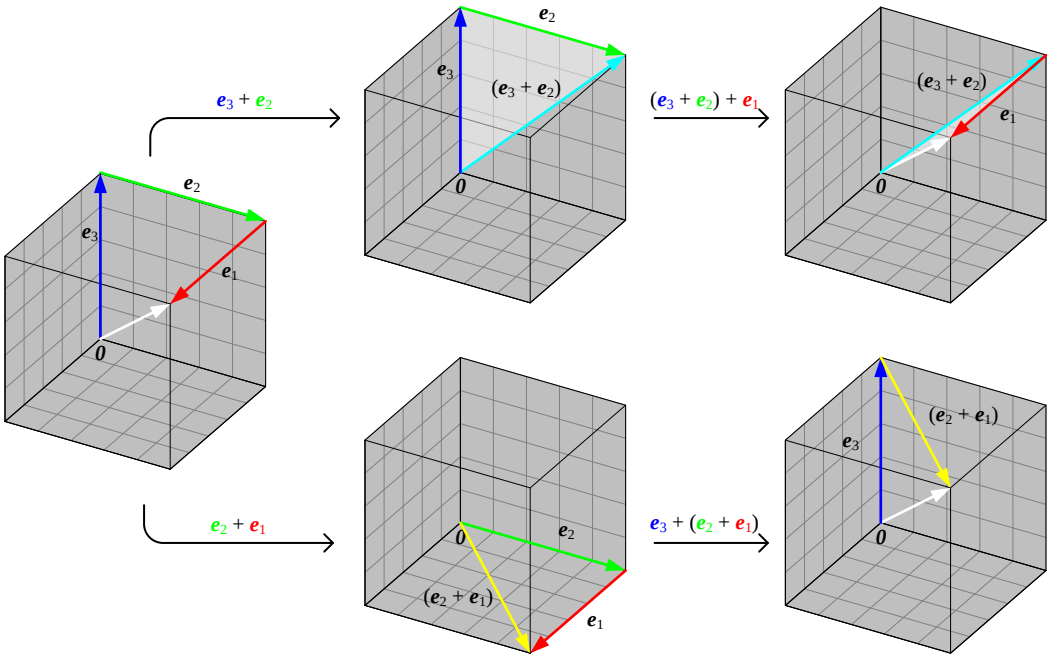

Figure 15. The Two Paths $(\boldsymbol{e}_3 + \boldsymbol{e}_2) + \boldsymbol{e}_1$ and $\boldsymbol{e}_3 + (\boldsymbol{e}_2 + \boldsymbol{e}_1)$

Think about which other orderings of the red vector $\boldsymbol{e}_1$, green vector $\boldsymbol{e}_2$, and blue vector $\boldsymbol{e}_3$ can also be combined to reach the white vector.

Subtraction

Vector subtraction is the component-wise subtraction of two vectors of the same dimension. For the n-dimensional column vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ given earlier in this section, their difference is

$$\boldsymbol{a}-\boldsymbol{b}=\begin{bmatrix}a_1\\a_2\\\vdots\\a_n\end{bmatrix}-\begin{bmatrix}b_1\\b_2\\\vdots\\b_n\end{bmatrix}=\begin{bmatrix}a_1-b_1\\a_2-b_2\\\vdots\\a_n-b_n\end{bmatrix}$$

For example, in RGB color space, subtracting a color vector means filtering out that color, as in the three examples below.

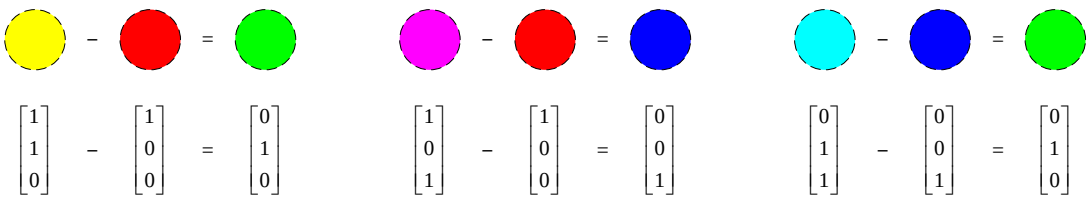

Figure 16. Vector Subtraction in RGB Color Space

With vector addition in hand, vector subtraction becomes intuitive and natural to understand. The subtraction above can be written in the following additive form

$$\boldsymbol{a}-\boldsymbol{b}=\boldsymbol{a}+(-\boldsymbol{b})=\begin{bmatrix}a_1\\a_2\\\vdots\\a_n\end{bmatrix}+\begin{bmatrix}-b_1\\-b_2\\\vdots\\-b_n\end{bmatrix}=\begin{bmatrix}a_1-b_1\\a_2-b_2\\\vdots\\a_n-b_n\end{bmatrix}$$

where $-\boldsymbol{b}$ is called the opposite vector of $\boldsymbol{b}$, also known as its additive inverse. This foreshadows scalar multiplication, a topic developed in the next section: the opposite vector is simply the original vector scaled by the number −1.

Figure 17 shows the sum of $\boldsymbol{a}$ and $-\boldsymbol{b}$ computed using the parallelogram law.

Geometrically, $-\boldsymbol{b}$ and $\boldsymbol{b}$ have the same length but opposite directions. Thus, the vector subtraction $\boldsymbol{a}-\boldsymbol{b}$ can be understood as starting from the origin, first moving along $\boldsymbol{a}$, then moving along $-\boldsymbol{b}$, and finally arriving at the position $\boldsymbol{a}-\boldsymbol{b}$.

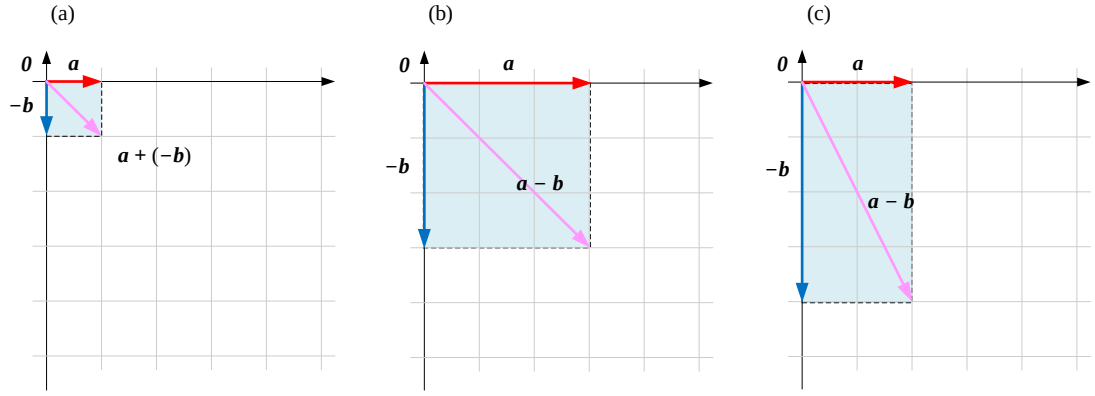

Figure 17. A Few Examples of Computing the Vector Subtraction $\boldsymbol{a}-\boldsymbol{b}$ Using the Parallelogram Law

From another perspective, let

$$\boldsymbol{c}=\boldsymbol{a}-\boldsymbol{b}$$

Adding $\boldsymbol{b}$ to both sides of the equation gives

$$\boldsymbol{b}+\boldsymbol{c}=\boldsymbol{a}$$
$$\boldsymbol{b}+(\boldsymbol{a}-\boldsymbol{b})=\boldsymbol{a}$$

This means that $\boldsymbol{b}$ and $\boldsymbol{a}-\boldsymbol{b}$ sum to $\boldsymbol{a}$.

As shown in Figure 18, aligning the starting points of vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ at $\boldsymbol{0}$, $\boldsymbol{a}-\boldsymbol{b}$ has as its result the segment from the endpoint of $\boldsymbol{b}$ to the endpoint of $\boldsymbol{a}$. This, fundamentally, is again an instance of the triangle law.

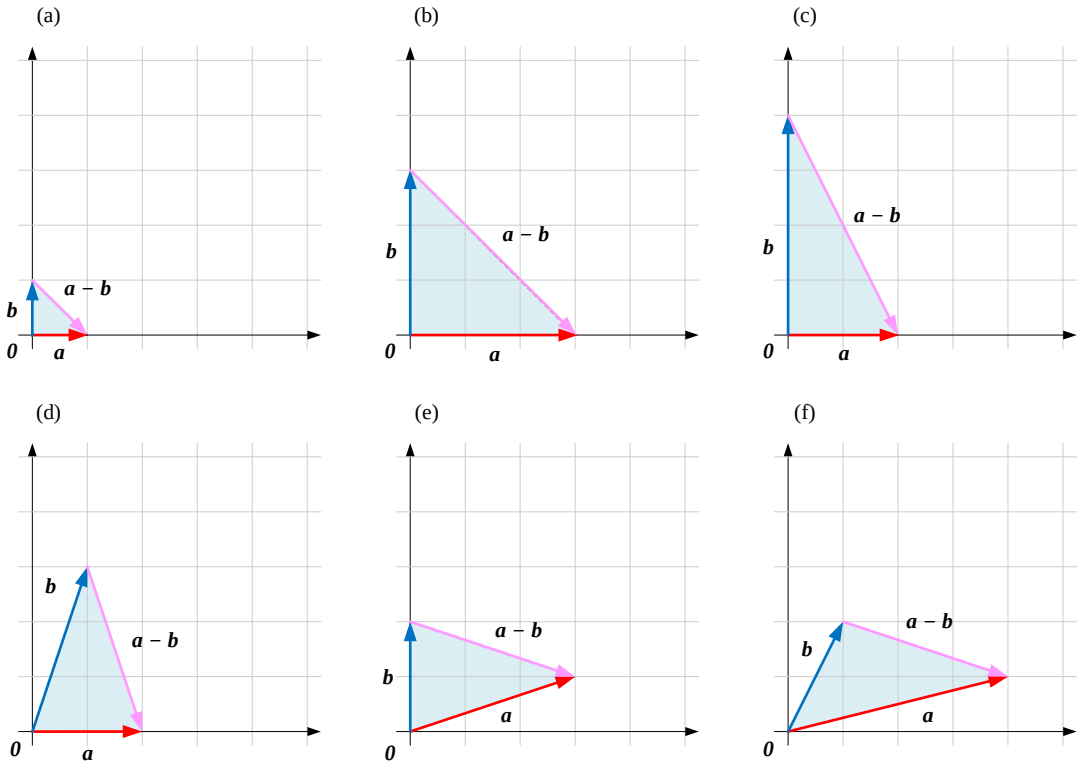

Figure 18. A Few Examples of Computing the Vector Subtraction $\boldsymbol{a} - \boldsymbol{b}$ Using the Triangle Law

Please use tools such as DeepSeek/ChatGPT to complete the following exercises for this section.

Q1. Use np.array() to create the following vectors and compute their sum.

$\boldsymbol{a} = [1, 2, 3]^T$, $\boldsymbol{b} = [3, 4, 5]^T$, $\boldsymbol{c} = [5, 6, 7]^T$

Q2. Use Code 2 to visualize each subplot of Figure 3.

Q3. Learn to use the function below, and modify Code 2, to add a fill color to the parallelogram.

https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.fill.html

Q4. Modify Code 1 to compute the vector subtractions $\boldsymbol{a} - \boldsymbol{b}$ and $\boldsymbol{b} - \boldsymbol{a}$.

$\boldsymbol{a} = [3, 2, 1]^T$, $\boldsymbol{b} = [5, 4, 3]^T$

Q5. Write Python code to plot Figure 8.

Q6. Write Python code to plot Figure 13.

Q7. Use the parallelogram law to visualize Figure 3 subplots (d), (e), (f), where $\boldsymbol{a} - \boldsymbol{b}$.

Q8. Based on Figure 18, draw $\boldsymbol{b} - \boldsymbol{a}$.